

Longest Common Subsequence and Edit Distance

Programming and Data Structures — Week 9, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state the longest common subsequence problem precisely, explicitly distinguishing it from substring matching, which requires contiguous characters; build and read a complete LCS dynamic programming table by hand, and reconstruct the actual subsequence achieving the optimal length, not merely its length; state the edit distance problem and recognize its recurrence as LCS's recurrence with one additional option; and explain how both problems connect directly to real tools — the Unix `diff` utility, spell-checker suggestion algorithms, and DNA sequence alignment in bioinformatics.

Outline

Today covers a two-editors'-diff analogy motivating the LCS problem; a precise statement of LCS, explicitly distinguished from substring matching; the LCS recurrence, derived directly from first principles rather than presented as a given formula; building the complete LCS table by hand on a concrete example; reconstructing the actual subsequence, not just its length; edit distance, whose recurrence adds a single new option on top of LCS's; the MTech-track connections to `diff`, spell-checker suggestions, and DNA sequence alignment; an in-class quiz; and a summary.

The two-editors'-diff analogy

Picture two people independently editing a shared document, who later need a tool to merge their changes and highlight what actually differs. A `diff` tool identifies the changes by finding the longest stretch of text both versions still share, in the same relative order, even if that shared text is scattered non-contiguously across the document — anything not part of that shared stretch is flagged as an insertion or deletion. This “longest shared sequence, in order, not necessarily contiguous” is precisely the longest common subsequence problem this lecture formalizes.

LCS, defined precisely — and why it is not substring matching

A subsequence of a string is any sequence obtainable by deleting zero or more characters, without reordering the characters that remain. "ACE" is a genuine subsequence of "ABCDE" — obtained by deleting B and D — but it is not a substring, since a substring specifically requires contiguous characters, and A, C, E are not adjacent in the original string. The longest common subsequence of two strings is the longest sequence that is simultaneously a subsequence of both — a strictly more flexible notion of “shared content” than requiring an exact contiguous match.

Tracing the distinction on a concrete example

```
s1 = "ABCBBDAB"      indices: A0 B1 C2 B3 D4 A5 B6
s2 = "BDCAB"          indices: B0 D1 C2 A3 B4
```

```
Longest common SUBSTRING (contiguous):      "AB", length 2
Longest common SUBSEQUENCE (gaps allowed): "BCAB", length 4
```

```
Matching positions for "BCAB":
s1: B(idx1), C(idx2), A(idx5), B(idx6)
s2: B(idx0), C(idx2), A(idx3), B(idx4)
```

Comparing "ABCBBDAB" and "BDCAB" directly shows the two notions diverge sharply: the longest common *substring* is short, just "AB" (length 2), since few contiguous runs match exactly in both strings. The longest common *subsequence*, "BCAB" (length 4, confirmed by direct computation), is considerably longer, since it allows the matching characters — B, then C, then A, then B — to be spread out with gaps between them in each original string, as long as their relative order is preserved. This gap-tolerance is exactly what makes LCS the right model for document diffing — real edits insert and delete text between the parts that remain unchanged.

Deriving the LCS recurrence from first principles

Deriving the recurrence rather than merely stating it: define $L(i, j)$ as the length of the LCS between the first i characters of s_1 and the first j characters of s_2 . If the last characters of both prefixes match, that matching character can always be included in some optimal LCS (a standard exchange argument justifies this, omitted here for brevity), reducing the problem to $1 + L(i - 1, j - 1)$ — one match, plus the best LCS of everything before it. If the last characters differ, the optimal LCS must exclude the final character of at least one of the two strings, and trying both options and taking the better, $\max(L(i - 1, j), L(i, j - 1))$, is guaranteed to find the true optimum, by the same optimal-substructure argument Lecture 2 used for knapsack.

Building the LCS table, completely, by hand

Building the complete table for $s_1 = \text{"ABC"}$ against $s_2 = \text{"AC"}$: the first row and column are 0, the base case of an empty prefix. Cell (A, A) matches, giving $1 + L(0, 0) = 1$. Cell (B, A) does not match ($B \neq A$), giving $\max(L(A, A), L(B, \text{""})) = \max(1, 0) = 1$. Cell (C, C) matches, giving $1 + L(AB, A) = 1 + 1 = 2$ — the final answer, confirming the LCS has length 2 (the subsequence "AC" itself, present in both strings).

Filling one cell, traced in complete detail

Filling $L(B, A)$ [row "B", column "A"]:

$s_1[\text{B's position}] = \text{'B'}$, $s_2[\text{A's position}] = \text{'A'}$. $\text{'B'} \neq \text{'A'} \rightarrow$ NOT a match
Option 1: $L(\text{row above, same column}) = L(\text{"A" prefix, "A" prefix}) = 1$
Option 2: $L(\text{same row, column to left}) = L(\text{"B" prefix, "" prefix}) = 0$
 $\max(1, 0) = 1 \rightarrow L(B, A) = 1$

Tracing a single non-matching cell in complete detail: filling the entry for s_1 prefix ending in B against s_2 prefix ending in A finds a mismatch, since B differs from A. The recurrence then checks two options — the cell directly above (excluding B from s_1 's consideration) and the cell directly to the left (excluding A from s_2 's consideration) — taking the maximum of the two, which here is 1. This exact two-option check, generalized across the whole table, is the entire LCS algorithm.

Reconstructing the actual subsequence

```
def reconstruct_lcs(table, s1, s2, i, j):
    if i == 0 or j == 0:
        return ""
    if s1[i-1] == s2[j-1]:
        return reconstruct_lcs(table, s1, s2, i-1, j-1) + s1[i-1]
    elif table[i-1][j] >= table[i][j-1]:
        return reconstruct_lcs(table, s1, s2, i-1, j)
    else:
        return reconstruct_lcs(table, s1, s2, i, j-1)
```

Reconstructing the actual subsequence, not just its length, walks the table backward from the final cell — exactly Lecture 2's reconstruction technique, applied to a different DP table. Whenever the current characters match, that character was genuinely part of the LCS, and the walk moves diagonally to the cell representing both strings with that character removed. Whenever they do not match, the walk follows whichever direction, up or left, the original recurrence actually used to reach the larger of the two values, retracing the exact decision made when the table was filled.

Reconstruction, traced completely on this lecture's table

Start at $(i=3, j=2) = L(C, C) = 2$.

$s1[2]='C', s2[1]='C' \rightarrow$ MATCH $\rightarrow$ record 'C', move to $(i=2, j=1)$

$(i=2, j=1) = L(B, A) = 1$.

$s1[1]='B', s2[0]='A' \rightarrow$ no match. $L(i-1, j)=L(A, A)=1$, $L(i, j-1)=L(B, "")=0$. $1 \geq 0 \rightarrow$ move to $(i=1, j=1)$

$(i=1, j=1) = L(A, A) = 1$.

$s1[0]='A', s2[0]='A' \rightarrow$ MATCH $\rightarrow$ record 'A', move to $(i=0, j=0)$

$(i=0, j=0)$: base case, stop.

Reconstructed (reversed order recorded): "A" then "C" $\rightarrow$ LCS = "AC"

Tracing reconstruction on the "ABC"/"AC" table built earlier: starting at the final cell, the characters match (C and C), so C is recorded and the walk moves diagonally. At the next cell, B and A do not match, and since the cell above (value 1) is at least as large as the cell to the left (value 0), the walk moves up. At that cell, A and A match, recording A and moving diagonally to the base case, where the walk stops. Reading the recorded characters in reverse order (since they were recorded last-to-first during the backward walk) gives "AC" — correctly matching the table's final value of 2, since "AC" has length 2.

Edit distance: LCS's recurrence, plus one more option

Edit distance, also called Levenshtein distance, measures the minimum number of single-character insertions, deletions, or substitutions needed to transform one string into another. Its recurrence closely mirrors LCS's: if the current characters match, no edit is needed at this position, and the cost is simply $D(i-1, j-1)$, carried forward unchanged. If they do not match, three options are considered — substituting (diagonal), deleting from s_1 (up), or inserting into s_1 to match s_2 (left) — each costing 1 plus whichever smaller subproblem it reduces to, and the minimum of the three is taken. This is LCS's exact two-option recurrence with one additional option (substitution) added, reflecting edit distance's slightly richer set of allowed operations.

The picture: LCS table versus edit distance table, side by side

LCS table ("ABC" vs "AC")				Edit distance table (shape only)	
	""	A	C	Same grid shape, same fill order — but MINIMIZES cost, and mismatches check THREE neighbors (up, left, AND diagonal), not two.	
""	0	0	0		
A	0	1	1		
B	0	1	1		
C	0	1	2		

maximize length; matches add 1 diagonally

Placing both tables side by side on the same input pair makes the shared structure and the one genuine difference visible directly: both are filled cell by cell using only already-computed neighbors, but LCS maximizes a length while edit distance minimizes a cost, and edit distance's mismatch case considers one additional direction (diagonal) that LCS's mismatch case does not use.

MTech addition — connecting to real tools

These are not merely textbook exercises. The Unix `diff` utility computes something extremely close to the longest common subsequence between two files, treated line by line, to produce the minimal patch describing their differences — the exact mechanism this lecture built from scratch. Spell-checker suggestion algorithms rank candidate corrections for a misspelled word by their edit distance from it, favoring smaller distances as more likely intended corrections. DNA sequence alignment in bioinformatics uses LCS and edit distance, often with biologically motivated cost variants (a substitution might cost differently than an insertion, reflecting real mutation rates), to measure how closely two gene sequences correspond — a genuinely major, real-world application of exactly this lecture's algorithms.

Edit distance, one cell traced completely

Tracing one cell of the edit-distance table for transforming "cat" into "cot", at the position comparing prefixes "ca" and "co": the final characters a and o mismatch, so all three options are considered. Substitution costs $1 + D("c", "c") = 1 + 0 = 1$, since "c" trivially matches "c" with zero edits. Deletion and insertion each cost $1 + 1 = 2$, using previously computed smaller subproblems. The minimum, 1, correctly reflects that transforming "cat" to "cot" needs only a single substitution — a to o — the cheapest of the three options at this position.

Exercise

As an exercise, build the complete LCS table by hand for "AGCAT" and "GAC", and use this lecture's reconstruction technique to identify the actual longest common subsequence. Separately, implement `edit_distance` and confirm it returns 3 for "kitten" and "sitting", then trace by hand which three specific edits — substitutions, insertions, or deletions — achieve this minimum.

Cost of both algorithms

Both LCS and edit distance share the identical cost structure: a table of size $(m + 1) \times (n + 1)$ for strings of length m and n , with each cell costing $O(1)$, giving $O(mn)$ total — the same shape of result as Lecture 2's $O(nW)$ knapsack cost, polynomial in the problem's natural dimensions. A naive recursive approach exploring every possible way to align two strings would cost exponentially more, repeating exactly the overlapping-subproblems pattern Lectures 1 and 2 already established for Fibonacci and knapsack respectively — the same underlying phenomenon, recognized here in a third, genuinely different problem domain.

Quiz — LCS or substring?

Given the strings "ABCDE" and "ACE", determine whether "ACE" qualifies as a common subsequence, a common substring, both, or neither, before checking the answer.

Quiz — answer

"ACE" is trivially both a subsequence and a substring of itself, since it is identical to the second string. Within "ABCDE", "ACE" appears with B and D skipped between its characters — a valid subsequence, since order is preserved even with gaps — but it is not a substring of "ABCDE", since the three characters are not contiguous there. The correct answer is that "ACE" is a common subsequence of the two strings, but not a common substring, directly illustrating the distinction this lecture opened with.

A closing note: LCS length vs. LCS *a* solution

A precise closing distinction worth stating explicitly: the LCS *length* is always unique, since the table's final cell holds exactly one numeric value. The LCS *string* achieving that length, however, may not be unique at all — multiple genuinely different subsequences can share the same maximum length, particularly when a mismatch cell's two options (up versus left) tie exactly. This lecture's reconstruction algorithm returns one valid optimal subsequence, using a fixed tie-breaking rule, but it is not necessarily the only correct answer — worth stating explicitly whenever reporting a specific reconstructed result, rather than implying uniqueness that does not actually hold.

Summary

Longest common subsequence finds the longest shared sequence between two strings, allowing gaps, genuinely distinct from substring matching, which requires contiguous characters. Its recurrence — a match contributes 1 plus the diagonal value, a mismatch takes the maximum of the cell above or to the left — was derived from an explicit optimal-substructure argument, not merely stated. Reconstruction, using the same backward-walk technique from Lecture 2, recovers the actual subsequence achieving the optimal length. Edit distance extends this exact recurrence with one additional option, substitution, measuring transformation cost rather than shared length, and both problems underpin real tools — `diff`, spell-checkers, and DNA sequence alignment. Next lecture closes this unit by consolidating every dynamic programming problem covered and building the general pattern-recognition skill needed to identify DP opportunities in genuinely new problems.